CHAPTER 11 · BOOKS API

JWT Authentication

in Books REST API

Slim Framework · Firebase JWT · PHP 8



What is JWT?

JSON Web Token — a compact, self-contained way to transmit trusted information

HEADER

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Algorithm & token type

PAYLOAD

```
{
  "iss": "books-api",
  "sub": 3,
  "role": "admin",
  "iat": 1700000000,
  "exp": 1700003600
}
```

Claims — user data

SIGNATURE

```
HMAC-SHA256(
  base64(header) +
  "." +
  base64(payload),
  SECRET
)
```

Integrity proof

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 . eyJpc3MiOiJib29rcy1hcGkiLCJzdWIiOjMsInJvbGUiOiJhZG1pbiJ9 . SflKxwRJSMeKKF2QT4fwpM

JWT Configuration — JwtService.php

```php
namespace App\Auth;
use Firebase\JWT\JWT;
use Firebase\JWT\Key;

final class JwtService {
    private string $secret;
    private string $algo = 'HS256';
    private int    $ttl;
    private string $issuer;

    public function __construct() {
        $this->secret = $_ENV['JWT_SECRET'];
        $this->ttl    = (int)($_ENV['JWT_TTL'] ?? 3600);
        $this->issuer = $_ENV['JWT_ISSUER'] ?? 'books-api';
    }
}
```

🔑 **JWT_SECRET**

`Your signing key`

Never committed to Git

⏱️ **JWT_TTL**

`3600  (1 hour)`

Token lifetime in seconds

🏷 **JWT_ISSUER**

`"books-api"`

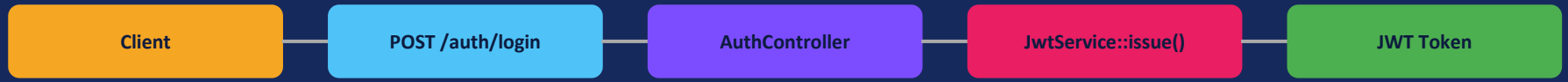
Identifies the token creator

🔐 **Algorithm**

`HS256 (HMAC-SHA256)`

Symmetric — one shared secret

Step 1 — Token Issuance (Login)



JWT Payload Claims

`iss`	`"books-api"`	*Issuer — who created the token*
`sub`	`3`	*Subject — user ID*
`iat`	`1700000000`	*Issued-At timestamp*
`exp`	`1700003600` (+1 hr)	*Expiry timestamp*
`role`	`"admin" | "member"`	*User role (extra claim)*
`email`	`"user@example.com"`	*Email (extra claim)*

Response JSON

```
{
  "token_type":   "Bearer",
  "expires_in":   3600,
  "access_token": "eyJhbGci..."
}
```

The client stores this token and includes it
in the Authorization header for protected requests.

Step 2 — Client Sends the Token

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJib29rcy1hcGkiLCJzdWIiOjMsInJvbGUiOiJhZG1pbiJ9.xxxx
```

🌐 **PUBLIC (No Token Required)**

```
GET    /api/books

GET    /api/books/{id}

POST   /auth/register

POST   /auth/login
```

🔒 **PROTECTED (Bearer Token)**

```
POST   /api/books

PUT    /api/books/{id}

DELETE /api/books/{id}  + admin

GET    /auth/me
```

Step 3 — AuthMiddleware Verification

1 **Extract token from header**

```
$hdr = $req->getHeaderLine('Authorization');
if (!preg_match('/^Bearer\s+(.+)$/i', $hdr, $m)) {
    return $this->fail('Missing or malformed token'); // 401
}
```

2 **Verify signature & expiry**

```
try {
    $payload = $this->jwt->verify($m[1]);
} catch (\Throwable $e) {
    return $this->fail('Invalid or expired token'); // 401
}
```

3 **Attach payload to request**

```
$req = $req->withAttribute('auth', $payload);
return $h->handle($req); // pass to controller
```

⚠ Invalid / expired token → 401 Unauthorized + WWW-Authenticate: Bearer header returned

Step 4 — Using the Payload in Controllers

No extra database call needed — identity lives inside the token

BookController::delete() — Admin role check

```
$auth = (array)$req->getAttribute('auth', []);
if (($auth['role'] ?? 'member') !== 'admin') {
    return $this->json($res, ['error' => 'Admins only'], 403); // Forbidden
}
```

AuthController::me() — Fetch profile from token sub

```
$auth = (array)$req->getAttribute('auth', []);
$u = $this->users->findById((int)($auth['sub'] ?? 0));
return $u ? $this->json($res, $u) : $this->json($res, ['error' => 'Not found'], 404);
```

Protected vs Public Routes

Route	Method	Auth Required	Extra Condition
/api/books	GET	No	—
/api/books/{id}	GET	No	—
/auth/register	POST	No	—
/auth/login	POST	No	—
/api/books	POST	Bearer	—
/api/books/{id}	PUT	Bearer	—
/api/books/{id}	DELETE	Bearer	role = admin
/auth/me	GET	Bearer	—

Key Design Decisions

🔐 HS256 — Symmetric signing

One shared secret for both signing and verification. Ideal for a single-server API where key distribution complexity is unnecessary.

⚡ Stateless architecture

No session table or server-side storage needed. Any request can be validated purely from the token — scales horizontally.

⚙ TTL configured via .env

Token lifetime adjustable per environment (dev/staging/prod) without touching source code. Default: 3600 s.

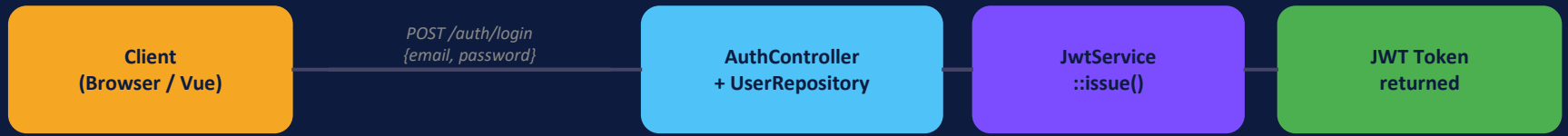
🔗 withAttribute('auth', payload)

Slim's clean pattern for passing verified data downstream. No global state, no superglobals, fully testable.

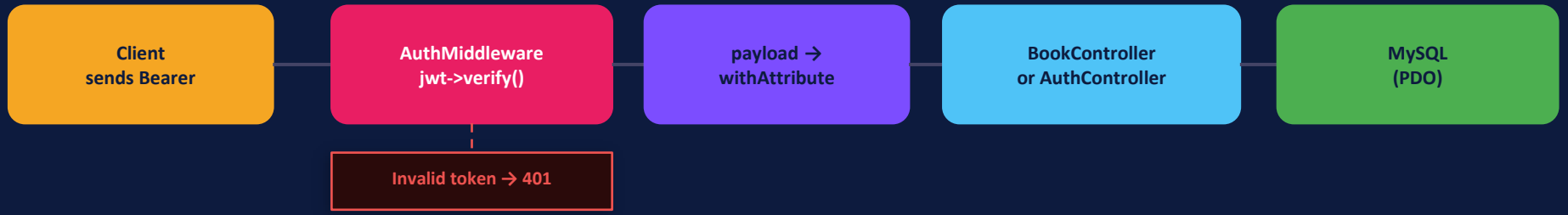
👤 Role stored inside the token

admin vs member enforced in controllers from the payload. Zero extra database calls per request for authorization.

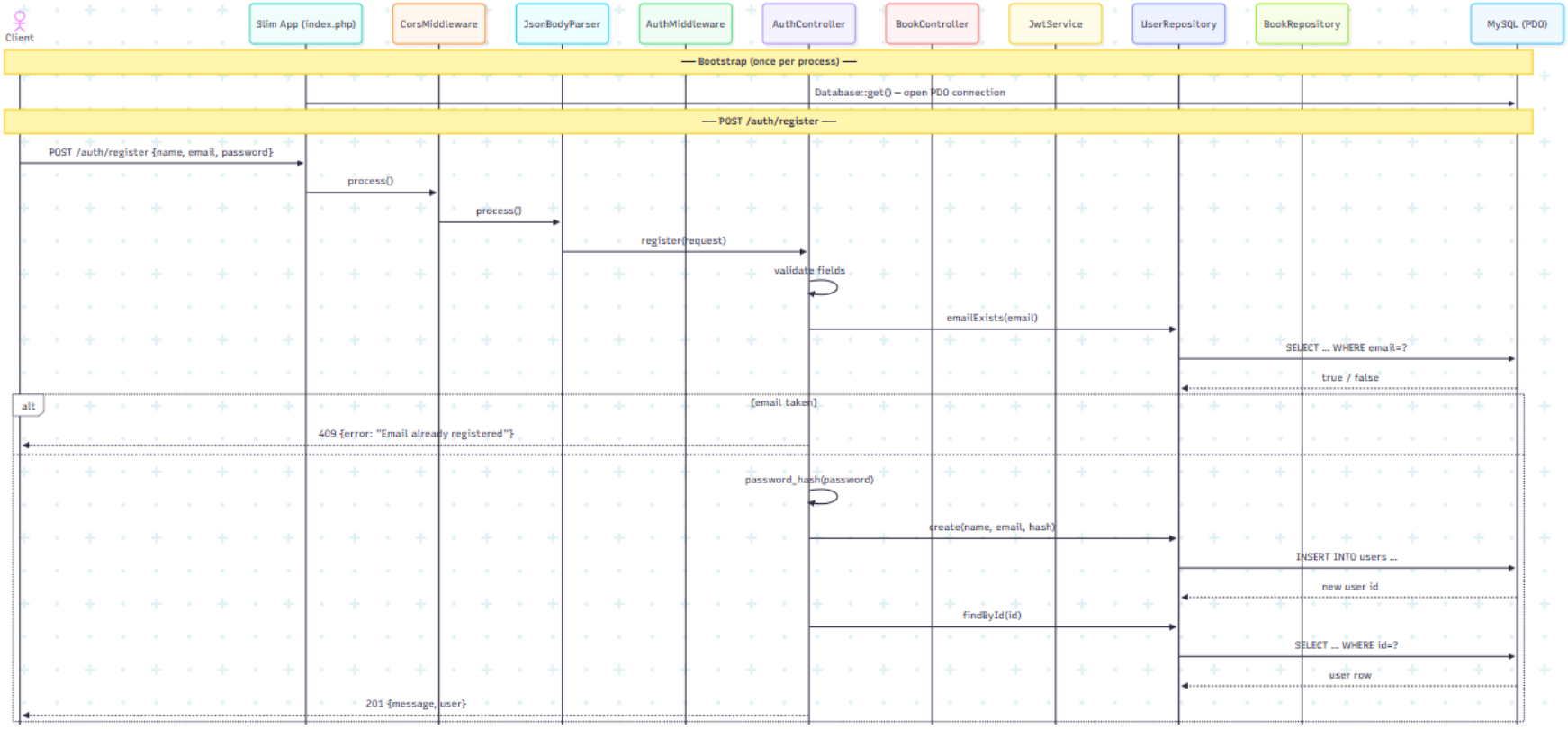
Complete JWT Flow



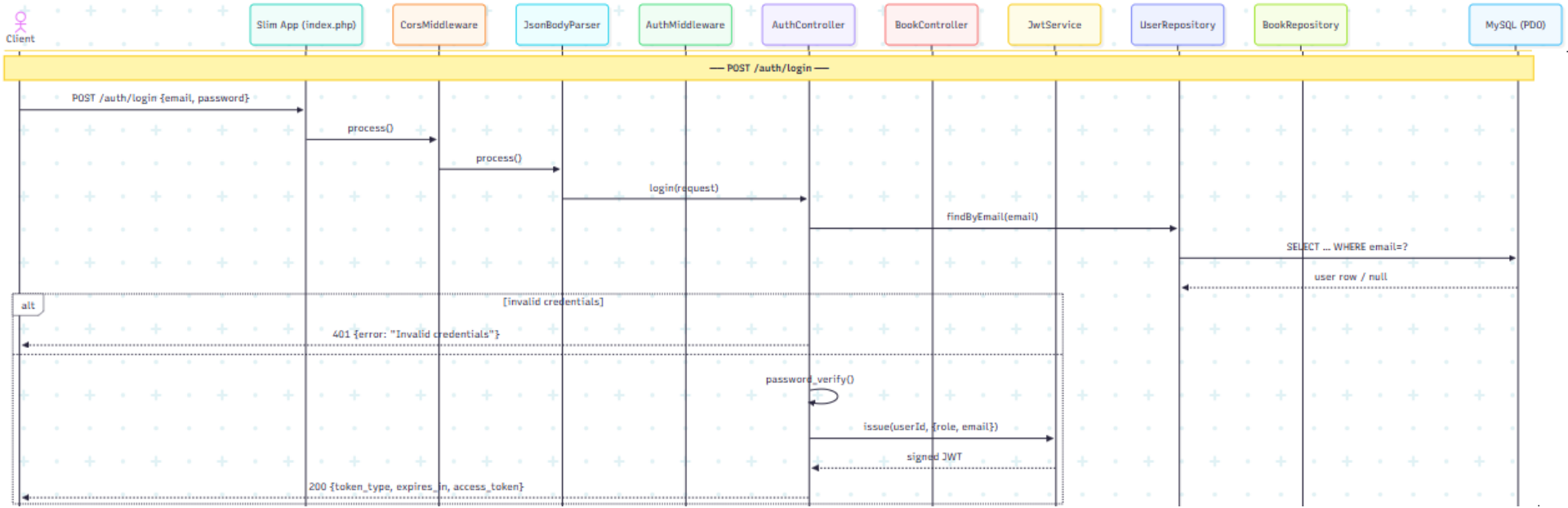
Protected Request Flow



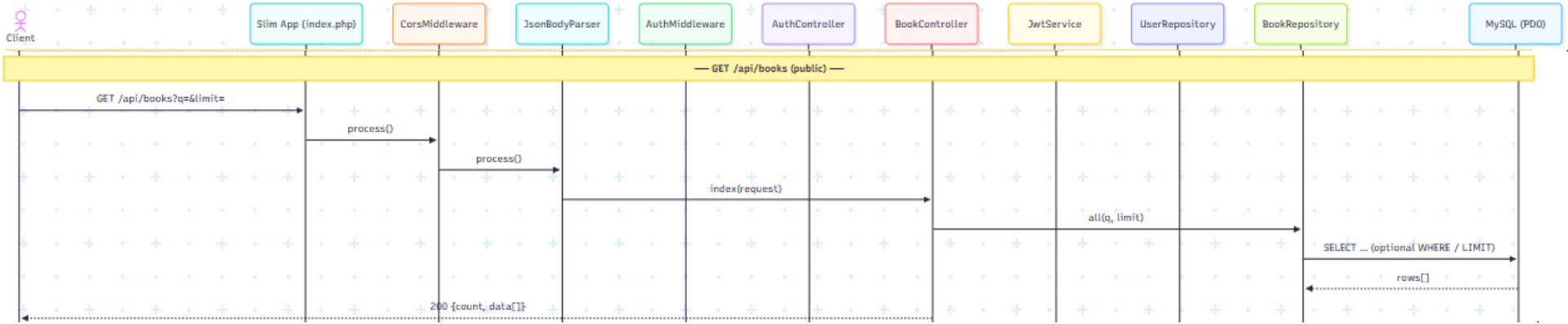
POST /auth/register



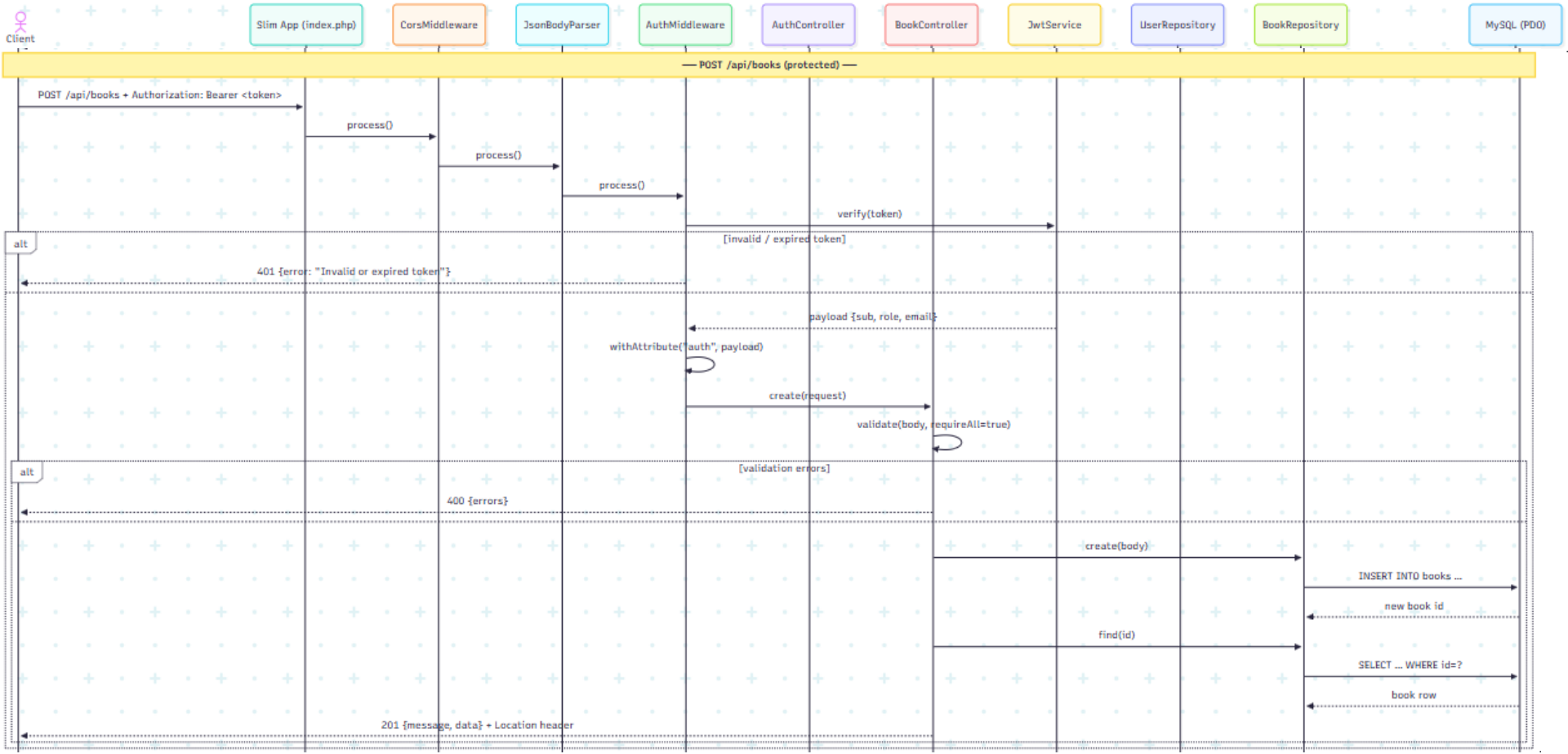
POST /auth/login



GET /api/books (public)



POST /api/books (protected)



DELETE /api/books/{id} (admin only)

